

Web Applications

Clousers, Currying, Destructuring, Spread Operator, Rest Operator, and Generators

Suhel Hammoud

■ Advanced JavaScript Features

- Closures
- Currying
- Destructuring Parameters
- Spread Operator
- Rest Operator
- Generators

These features enhance expressiveness, modularity, and control flow in modern JavaScript.

Closures: Capturing State

A **closure** is created when a function "remembers" the variables from its outer scope, even after the outer function has finished executing.

  Example:

```
function makeCounter() {  
  let count = 0;  
  return function () {  
    return ++count;  
  };  
}  
  
const counter = makeCounter();  
console.log(counter()); // 1  
console.log(counter()); // 2
```

✓ **count** persists across function calls – that's a closure!

Closures: Real-World Uses

- Data encapsulation
- Function factories
- Maintaining private state
- Event handlers and callbacks

Tip:

Closures are foundational in functional and asynchronous programming in JavaScript.

Currying: Breaking Functions into Steps

Currying is the process of transforming a function with multiple parameters into a sequence of functions, each taking one parameter.

 Example:

```
function multiply(a) {  
  return function (b) {  
    return a * b;  
  };  
}  
  
const double = multiply(2);  
console.log(double(5)); // 10
```

✓ Each function remembers its argument through a closure.

Currying: With Arrow Functions

You can write curried functions concisely using ES6 arrow syntax:

```
const add = a => b => a + b;  
  
console.log(add(3)(4)); // 7
```

💡 Why Curry?

- Enables **function reuse**
- Useful in **functional composition**
- Cleaner **partial application** of arguments

📦 Destructuring Parameters: Unpacking Values

Destructuring allows extracting values from arrays or objects and assigning them to variables in a single step.

📋 🖋️ Array Example:

```
const [x, y] = [10, 20];  
console.log(x, y); // 10 20
```

📋 🖋️ Object Example:

```
const user = { name: "Alice", age: 30 };  
const { name, age } = user;  
console.log(name, age); // Alice 30
```

Destructuring Function Parameters

You can destructure directly in the function signature:

```
function greet({ name, age }) {  
  console.log(`Hello ${name}, age ${age}`);  
}  
  
greet({ name: "Bob", age: 25 });
```

✓ Clean and concise for objects with multiple keys

💡 Also works in array-based function params

The Spread Operator (...)

The **spread operator** expands an array (or object) into individual elements.

Examples:

```
const nums = [1, 2, 3];  
console.log(...nums); // 1 2 3  
  
const arr1 = [1, 2];  
const arr2 = [...arr1, 3, 4];  
console.log(arr2); // [1, 2, 3, 4]  
  
const obj1 = { a: 1, b: 2 };  
const obj2 = { ...obj1, c: 3 };  
console.log(obj2); // { a: 1, b: 2, c: 3 }
```

Useful for:

- Cloning arrays/objects
- Concatenation
- Passing arguments to functions

The Rest Operator (...)

The **rest operator** collects remaining items into a single array or object.

Function Parameters:

```
function sum(...nums) {  
  return nums.reduce((a, b) => a + b, 0);  
}  
console.log(sum(1, 2, 3)); // 6
```

Destructuring:

```
const [first, ...rest] = [10, 20, 30, 40];  
console.log(first); // 10  
console.log(rest);  // [20, 30, 40]
```

✓ Rest syntax helps capture "the rest" of a list or arguments cleanly.

💡 Spread **expands**, Rest **gathers** – same syntax, opposite purpose depending on context.

Destructuring Function Parameters

You can destructure directly in the function signature: f

```
let reverse = ([x, ...y]) => y.length > 0 ? [...reverse(y), x] : [x];
```

✅ Clean and concise for objects with multiple keys

💡 Also works in array-based function params

Generators: Pausable Functions

Generators are special functions that can **pause and resume** their execution using **yield**.

Syntax:

```
function* generatorFunc() {  
  yield 1;  
  yield 2;  
  yield 3;  
}
```

- Use ***** in function declaration
- Use **yield** to return values incrementally
- Call **.next()** to resume execution

Using Generators

```
function* idGenerator() {  
  let id = 1;  
  while (true) {  
    yield id++;  
  }  
}
```

```
const gen = idGenerator();  
  
console.log(gen.next().value); // 1  
console.log(gen.next().value); // 2  
  
const [a, b, , c, d] = idGenerator();  
console.log(a, b, c, d); // 0 1 3 4
```

✓ Generators are useful for:

- Iterators
- Lazy evaluation
- Asynchronous control flows

Summary

Feature	Description	Use Case
Closures	Functions that retain access to outer scope	Data hiding, stateful logic
Currying	Break a function into chained calls	Reuse, function composition
Destructuring	Extract values from objects/arrays	Clean code, easy parameter access
Spread Operator	Expands arrays/objects into individual elements	Merging, cloning, argument passing
Rest Operator	Collects multiple items into a single array/object	Variadic functions, flexible inputs
Generators	Functions that pause with <code>yield</code>	Iterators, async control flow

References

- `https://developer.mozilla.org`